AI Skin Care Service

"Mirror Me"

20232346 유소희 | 2026 캡스톤 디자인

Algorithm Design Role · OpenCV / YOLOv8 / Python

2. 프로젝트 전체 구조 한눈에 보기

아래 표가 코드 전체의 지도입니다.

단계	함수명	하는 일	연관 개념
1	preprocess()	이미지 밝기 보정 (CLAHE)	색공간(LAB), 히스토그램 평활화
2	extract_roi_*	얼굴 영역(ROI) 잘라내기	bbox, 좌표계
3	detect_redness()	홍조(빨간 영역) 감지	HSV 색공간, 마스킹
4	detect_pigmentation()	색소침착(반점) 감지	적응형 이진화, 모폴로지
5	detect_pore()	모공 크기 추정	Laplacian 분산
6	detect_sebum()	피지(빛 반사) 감지	밝기 임계값 처리
7	normalize()	원시값 → 0~100점 변환	정규화, MAX_VALUES 캘리브레이션
8	parse_label_files()	JSON 라벨 파일 읽기	AI Hub 데이터셋 구조, facepart
9	compare()	장비 측정치 vs 알고리즘 비교	오차율, 캘리브레이션
10	visualize()	결과 그래프 시각화	matplotlib, bar chart

3. 사용 라이브러리(모듈) 개요

코드 첫 줄에 import로 불러오는 도구들입니다.

라이브러리	역할	대안	왜 이걸 선택했나?
cv2 (OpenCV)	이미지 읽기, 색 변환, 필터링 등 컴퓨터 비전 전반	Pillow, scikit-image	실시간 처리 속도가 가장 빠르고, 컴퓨터 비전 함수가 가장 풍부함
numpy	이미지 = 숫자 배열. 픽셀 단위 수학 연산	list(기본 파이썬)	OpenCV가 내부적으로 numpy 배열을 사용하므로 필수. 연산 속도도 수십 배 빠름
matplotlib	결과 그래프(bar chart) 시각화 및 이미지 저장	seaborn, plotly	가장 보편적인 파이썬 시각화 라이브러리. 학술·연구 표준
json	AI Hub 라벨 파일 (.json) 읽기·파싱	xml, csv	AI Hub 데이터셋이 JSON 형식으로 제공되므로 선택의 여지 없음
os	파일 존재 여부 확인 (os.path.exists)	pathlib	os는 파이썬 내장 모듈이라 별도 설치 불필요. 간단한 파일 체크에 충분

4. preprocess() — 이미지 전처리

분석 전 이미지를 균일하게 보정하는 단계입니다.

4-1. 색공간 변환: BGR → LAB

색공간(Color Space)이란 색을 수치로 표현하는 방식입니다. 코드에서는 세 가지 색공간이 등장합니다.

색공간	구성 요소	특징	코드에서 사용 목적
BGR	Blue / Green / Red	OpenCV 기본값. 사람 눈 기반	이미지 기본 형식, 홍조 감지 기반
LAB	L(밝기) / A(녹↔빨) / B(파↔노)	밝기(L)와 색(A,B)이 분리됨 → 조명 영향 최소화	CLAHE 적용 (밝기만 보정)
HSV	Hue(색조)/Saturation(채도)/Value(밝기)	"빨간색"처럼 색을 범위로 지정하기 쉬움	홍조(붉은 영역) 감지 시 사용

4-2. CLAHE — 핵심 전처리 기법

CLAHE(Contrast Limited Adaptive Histogram Equalization)는 이미지의 밝기 대비를 지역별로 균일하게 만드는 알고리즘입니다.

구분	일반 히스토그램 평활화(HE)	CLAHE (이 코드에서 선택)
작동 방식	이미지 전체를 한꺼번에 보정	이미지를 작은 타일로 나눠 각각 보정
단점	피부 밝은 부분이 과하게 밝아지는 노이즈 발생	clipLimit로 과보정 방지
피부 분석 적합성	낮음 — 피부 디테일 왜곡 우려	높음 — 국소적 조명 차이 보정에 최적

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
# clipLimit: 대비 증폭 한계 (높을수록 더 강하게 보정, 노이즈 위험↑)
# tileGridSize: 이미지를 8×8 격자로 나눠 각 구역 독립 보정
```

Q. 왜 BGR을 바로 쓰지 않고 LAB으로 변환한 뒤 CLAHE를 적용했나요?

A. BGR에서 CLAHE를 적용하면 R, G, B 세 채널이 모두 영향을 받아 색상이 왜곡됩니다. LAB은 밝기(L 채널)가 색(A, B 채널)과 분리되어 있어, L 채널만 CLAHE로 보정하면 색상 변화 없이 밝기만 균일하게 만들 수 있습니다.

5. ROI 추출 — 얼굴 영역만 잘라내기

ROI(Region of Interest)란 분석에 필요한 관심 영역입니다. 배경을 제외하고 얼굴 피부 부위만 분석합니다.

5-1. 두 가지 ROI 추출 방식

구분	extract_roi_by_bbox() ← 우선 사용	extract_roi_center() ← 백업용
작동 방식	JSON의 bbox 좌표를 그대로 사용해 정확히 자름	이미지 중앙 50% 영역을 임시로 자름
언제 사용?	AI Hub JSON 파일이 있을 때 (정밀 분석)	JSON이 없을 때 임시 대체 (rough 분석)
정확도	높음 — 전문 장비로 측정된 좌표 사용	낮음 — 얼굴 위치가 중앙이 아닐 수 있음

🔀 **대안 비교 (YOLOv8/MediaPipe)** 실제 서비스에서는 YOLOv8이나 MediaPipe Face Mesh를 이용해 얼굴을 자동 탐지·분할합니다. 이 코드는 JSON bbox를 우선 사용하고, YOLOv8은 별도 모듈로 확장 예정이므로 현재는 단순 bbox 크롭 방식을 사용했습니다.

Q. extract_roi_center()가 백업용이라면, 실제 서비스에서는 어떻게 개선할 건가요?

A. YOLOv8 얼굴 탐지 모델을 연동해 자동으로 얼굴 bbox를 검출할 계획입니다. 현재 코드는 AI Hub JSON이 있는 개발·검증 환경 전용이고, 실제 사용자 업로드 이미지에는 YOLOv8 추론 결과의 bbox를 동일한 extract_roi_by_bbox()에 넣어 처리합니다.

6. 피부 특징 감지 — 4가지 핵심 알고리즘

6-1. detect_redness() — 홍조 감지

HSV 색공간에서 빨간색 범위(Hue 0~10°, 170~180°)에 해당하는 픽셀 비율을 계산합니다.

```
mask1 = cv2.inRange(hsv, (0,  50, 50), (10,  255, 255))   # 빨강 ①
mask2 = cv2.inRange(hsv, (170, 50, 50), (180, 255, 255)) # 빨강 ②
result = cv2.bitwise_or(mask1, mask2)                    # 두 마스크 합치기
```

6-2. detect_pigmentation() — 색소침착 감지

피부의 검은 반점(기미, 주근깨 등)을 감지합니다. 적응형 이진화(Adaptive Thresholding)와 모폴로지 연산을 사용합니다.

모폴로지 연산 (Morphological Operation)

이진화 후 노이즈(작은 점들)를 제거하는 후처리입니다.

```
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
# MORPH_OPEN = 침식(erosion) 후 팽창(dilation)
# 효과: 작은 노이즈 점 제거, 실제 반점 형태는 유지
```

📌 **파라미터 튜닝 근거** blockSize=51(큰 블록), C=20(높은 기준), 면적 80~1500px 필터링 — 모두 오탐(노이즈)을 줄이기 위한 의도적 강화입니다. 초기(blockSize=21, C=10, 30px)보다 엄격하게 조정해 진한 반점만 검출합니다.

6-3. detect_pore() — 모공 감지

Laplacian 변환의 분산(Variance)을 이용해 이미지의 "선명도"를 측정합니다. 모공이 클수록 경계가 뚜렷해 분산값이 높아집니다.

```
return float(cv2.Laplacian(roi_gray, cv2.CV_64F).var())
```

⚠ **한계 인정** 모공 실제 크기(픽셀 면적)와 Laplacian 분산의 단위가 다릅니다. 코드 내 compare() 함수에서 "단위가 달라서 점수로만 비교"라고 명시한 이유입니다. 향후 YOLOv8 개선할 예정입니다.

6-4. detect_sebum() — 피지 감지

피지가 많으면 피부가 번들거려 카메라에 밝게 반사됩니다. 밝기 200 이상인 픽셀의 비율을 피지 지표로 사용합니다.

```
_, thresh = cv2.threshold(roi_gray, 200, 255, cv2.THRESH_BINARY)
return float(np.count_nonzero(thresh)) / thresh.size
```

7. normalize() — 원시값을 0~100점으로 변환

```
MAX_VALUES = {
    "pigmentation": 200,    # 최대 200개의 반점까지 100점 범위로 매핑
    "pore":         20.0,   # Laplacian 분산 최대값
    "sebum":        0.6,    # 반사 비율 최대값
    "redness":      0.3,    # 붉은 픽셀 비율 최대값
}
```

```
def normalize(key, value):
    return min(100, int(value / MAX_VALUES[key] * 100))
```

📌 **MAX_VALUES 설정 기준** AI Hub 전문 장비 측정치와 알고리즘 결과를 비교(compare 함수)하여 실제 데이터 기반으로 보정한 값입니다. 예: pigmentation 오차 27.9%, pore 오차 11.5%로 검증.

점수 예시	pigmentation (반점 5개)	계산	결과
공식 적용	raw = 5개	5 / 200 × 100	2점 (좋음)
공식 적용	raw = 150개	150 / 200 × 100	75점 (나쁨)

Q. 점수가 낮을수록 좋은 건가요, 높을수록 좋은 건가요?

A. 이 코드에서 각 항목 점수(pigmentation, pore, sebum, redness)는 낮을수록 좋습니다(트러블이 적다는 의미). 종합 점수(total)는 100 - 평균으로 계산하므로 높을수록 좋습니다. visualize() 함수의 그래프 제목에도 "낮을수록 좋음"이라고 명시되어 있습니다.

8. parse_label_files() — AI Hub 데이터셋 파싱

AI Hub의 한국인 피부 데이터셋에는 전문 장비로 측정한 실제 수치가 JSON 파일로 제공됩니다. 이를 읽어 알고리즘 결과와 비교하는 기준값으로 사용합니다.

8-1. facepart — 얼굴 부위 번호 체계

번호	부위	코드에서 추출하는 정보	사용 목적
0	전체 얼굴	pigmentation_count (반점 개수)	색소침착 비교 기준
1	이마	forehead_moisture (수분)	피지 역산 기준
5	왼쪽 볼	l_cheek_pore, l_cheek_moisture	모공·수분 비교
6	오른쪽 볼	r_cheek_pore, r_cheek_moisture	모공·수분 비교

8-2. JSON 파싱 핵심 로직

```
with open(path, "r", encoding="utf-8") as f:
    d = json.load(f)
part = d["images"]["facepart"]   # 얼굴 부위 번호
bbox = d["images"]["bbox"]        # [x, y, w, h] 좌표
eq   = d.get("equipment") or {}  # 장비 측정값 딕셔너리
```

Q. 왜 JSON 파일을 8개나 사용하나요?

A. AI Hub 데이터셋은 얼굴 부위별로 별도 촬영한 이미지와 JSON을 제공합니다. 각 JSON에는 해당 부위의 장비 측정치만 있습니다. 전체 피부 분석을 위해 부위별 JSON을 합산·평균해야 하므로 여러 파일을 사용합니다.

9. compare() — 알고리즘 vs 전문 장비 검증

9-1. 오차율 계산

```
error = abs(algo - label)
rate  = round(error / max(label, 1) * 100, 1)    # % 오차율
ratio = algo / max(label, 1)                     # 배율 (과/미검출 판단)
```

9-2. MAX_VALUES 자동 튜닝 힌트

오차가 허용 범위(30%)를 벗어나면 MAX_VALUES 조정 힌트를 출력합니다.

조건	상황	권장 조치
ratio > 1.3	알고리즘이 장비보다 30% 이상 높게 검출 (과검출)	MAX_VALUES 값을 올려 점수 하향 조정
ratio < 0.7	알고리즘이 장비보다 30% 이상 낮게 검출 (미검출)	MAX_VALUES 값을 내려 점수 상향 조정
0.7 ~ 1.3	오차 허용 범위 내 (30% 이내)	현재 설정 유지

9-3. 현재 검증 결과 요약

항목	오차율	상태	비고
pigmentation	27.9%	허용 범위	반점 개수 직접 비교
pore	11.5%	허용 범위	점수 비교 (단위 상이)
sebum	2.1%	매우 우수	수분 역수 비교
redness	미검증	⚠ 추후 검증 필요	

Q. redness(홍조)는 왜 아직 검증이 안 됐나요?

A. AI Hub 데이터셋의 JSON에 홍조 관련 장비 측정치(redness 키)가 현재 샘플에 포함되지 않았습니다. 이후 여드름 (acne) 데이터셋으로 대체할 예정입니다.

10. visualize() — 결과 시각화

분석 결과를 두 가지 bar chart로 시각화하고 PNG 파일로 저장합니다. 이 부분이 프론트엔드와 직접 연결되는 지점입니다.

10-1. 그래프 구성

그래프	내용	의미
왼쪽	알고리즘 분석 점수 (pigmentation, pore, sebum, redness)	현재 피부 상태 — 낮을수록 좋음
오른쪽	장비 vs 알고리즘 오차율 (%)	알고리즘 신뢰도 검증 — 30% 기준선 표시

10-2. 프론트엔드 연동 방식

현재는 matplotlib으로 PNG 저장 → 프론트에서 이미지 표시 방식입니다. 실제 서비스에서는 아래 방식으로 개선됩니다.

단계	현재 (개발·검증용)	서비스 (목표)
데이터 전달	PNG 이미지 파일 저장	FastAPI → JSON scores 반환 → 프론트 수신
시각화	matplotlib (Python)	Chart.js / Recharts (JavaScript)
UI	없음	Figma 디자인 기반 대시보드 (꺾은선 + 레이더 차트)

11. analyze() — 전체 파이프라인 실행

순서	코드	하는 일	실패 시 처리
①	cv2.imread()	이미지 파일 읽기	None이면 FileNotFoundError 발생
②	preprocess()	CLAHE 조명 보정	—
③	parse_label_files()	JSON bbox + 장비 측정치 파싱	json_paths가 None이면 건너뜀
④	extract_roi_*()	얼굴 ROI 크롭	bbox 없으면 중앙 크롭으로 대체
⑤	detect_*()	4가지 피부 특징 수치 계산	—
⑥	normalize()	원시값 → 0~100점 변환	min(100, ...) 으로 100 초과 방지
⑦	compare()	장비 vs 알고리즘 오차 비교	ref 없으면 건너뜀
⑧	visualize()	bar chart 생성 · PNG 저장	—

13. 핵심 용어 사전 (발표 전 최종 점검)

용어	한 줄 정의
BGR	OpenCV의 기본 색공간. 빨강-녹색-파랑 순서가 반대인 것이 특징
LAB	밝기(L)와 색(A,B)이 분리된 색공간. 조명 보정에 최적
HSV	색조·채도·명도로 표현. 특정 색상 범위 검출에 유리
CLAHE	지역별 적응형 히스토그램 평활화. 조명 불균일 이미지 보정 알고리즘
ROI	Region of Interest. 분석 대상 관심 영역 (얼굴 부위 크롭)
bbox	Bounding Box. [x, y, w, h]로 표현하는 직사각형 영역
마스크(Mask)	특정 조건을 만족하는 픽셀만 흰색(255), 나머지 검은색(0)으로 표시한 이진 이미지
이진화	이미지를 흰색과 검은색 두 값만으로 변환하는 처리
모폴로지	침식·팽창 등 형태학적 연산으로 노이즈 제거 또는 객체 강조
Laplacian	이미지 경계(엣지) 강도를 측정하는 미분 필터. 분산이 클수록 선명
정규화	서로 단위가 다른 값들을 0~100 같은 동일 척도로 변환
facepart	AI Hub 데이터셋의 얼굴 부위 번호 (0=전체, 1=이마, 5=왼볼, 6=오른볼)
캘리브레이션	알고리즘 결과를 실제 장비 측정치에 맞게 조정하는 작업
YOLOv8	실시간 객체 탐지 딥러닝 모델. You Only Look Once의 8번째 버전
FastAPI	Python 기반 고성능 웹 API 프레임워크. OpenCV 분석 결과를 JSON으로 반환